

AIで防災アプリを作ってみて

作ってわかった、AIに任せられること・任せちゃいけないこと

開発メンバー勉強会 / 課題1

Kyoko Takazawa

AIは、作る速さを爆上げする。
でも「正しさ」の最後の一線は、
人が引くしかなかった。

— 防災アプリを一本作って、いちばん強く感じたこと

成果自慢は、しません

- 話すのは、作ってみて **感じたこと** の方
- 流れは、**課題** → **作ったもの** → **感じたこと** (本題)

「正解はない」課題です。ひとつの "やってみた記録" として聞いてください。

課題：アプリを一本、自分で作る

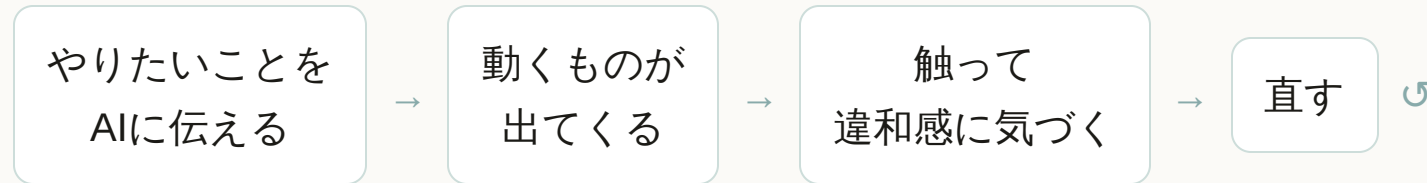
- 必須要件は **HTTPS** / **OIDC認証** / **Git管理**
- テーマは自由。何を作るかから、自分で決める

作ったのは「今すぐ、どこへ逃げる？」に答えるアプリ

- きっかけは、最近よく揺れること。
「そういえば、自分はどこに逃げればいいのかしら？」
- 既存のハザードマップは情報が多くて、とっさに答えをくれない

「見て考える」地図ではなく、「今の答え」をくれるものが欲しかった。

作り方：AIと "壁打ち" しながら



- 使ったのは **Claude Code** (ブラウザ版)。対話しながら進めた
- 正直、私はほとんどコードを書いていない。やっていたのは「問いを立てて、レビューする」こと

できたもの（詳しくは付録に）

- 災害の種類で、逃げる先が **変わる**
- 危険な浸水エリアを **避けて** 案内する
- 電波が無くても、一度見た街なら **動く**（オフライン対応）

公開URL <https://kyokoron.github.io/team-challenge/>

ここからが本題

作って "感じたこと"

AIは「それっぽい答え」を、自信満々に出す

- 最初 AI は「DBが要るのは投稿や更新のときだけ」と 言い切った
- 「それ、本当？」と 問い直したら、話が一気に深くなった
(オフライン・県境・更新・コスト...論点は山ほどあった)

鵜呑みにしなければ、AIは急に賢くなる。疑って問い直すのが、私の仕事だった。

「動く」と「正しい」は、
まったくの別物だった。

避難案内は、間違えると命に関わる。ここが一番の学び。

触ってみて、正直ゾッとした

内陸で「津波」を選ぶと海へ8km誘導
避難所が沿岸に偏っていて、逆方向へ導いていた

取得に失敗すると "架空の避難所" が出た
存在しない場所へ逃がしかねない

"最寄り" が最寄りじゃない
広域データで距離計算が狂い、順位が崩れる

直線距離を「徒歩〇分」と表示
川や線路で実際はもっと遠い = 油断を生む

これ全部、AIは "ちゃんと動いてる風" に出してきた。

だから「安全側に倒す」まで作り直した

- 危ないときは遠くへ誘導せず、「その場で上の階へ」と伝える
- 取れないデータは、偽物を出さず正直に止める
- 今いる場所が危険なら、最優先で警告する

"動く" はAIが作れる。**"間違えない" は、人が問い続けるしかなかった。 **

オープンデータは「ある」。でも「使える」まで遠い

- 避難所もハザードも、国のデータで揃う（しかも無料）
- でも実際に使うと...
 - どれが正解のデータか 分かりにくい
 - 洪水の想定は 河川ごとにバラバラで「街の1ファイル」が無い
 - 古い形式（Shapefile）で、変換・整形が必要

データを "使える形" に整える前処理が、実装と同じくらい重かった。

AI時代、価値の「置き場所」が変わる

これまで

ユーザー

↓ UIを操作

UI (アプリ)

↓

機能 (ロジック)

↓

データ

価値の中心：UI設計・機能開発・実装

これから

ユーザー

↓ 目的を伝える

AI (理解・推論)

↓

データ

価値の中心：①ドメイン理解／②良いデータ／③何を解くか

コードを書く力より、「何を解くべきか」を決める力。

今回、私がやっていたのは この3つ

① ドメインを理解する

「津波で海へ」の危うさに気づけたのは、コードでなく防災の理解だった

② 良いデータを用意する

オープンデータを "使える形" に整える。ここで品質が決まった

③ 何を解くかを決める

「動く」より「間違えない」。問いを立て直し、レビューし続けた

= 実装の速さはAIに任せた

私はこの3つに集中していた（と、後で気づいた）

後で知った：Anthropicも同じことを言っていた

「自分が何を知らないか」を明確にするほどAIは活
きる
まさに、問い直すほど質が上がった

「地図は現地ではない」
指示とAIの理解にはズレがある → 触って初めて気
づく、と重なった

自分の手探りが、公式の"コツ"と一致していて腑に落ちた。

参考：ナレッジセンス「Fable時代のAI活用法を、Anthropicの開発者が公開」(Zenn)

ちなみに、この資料も AI と作りました

- Marp = スライドを 文章（テキスト）で書く ツール
- だから AIが読める・書ける・直せる / Git で差分管理できる

「テキストで伝える → AIが形にする」時代に、資料もテキストで持てるのは相性がいい。
(ただし凝ったビジュアル勝負なら、Canva等が今も有利)

うまくいかなかったこと

- 動いてる風で、裏で "偽データ・誤った順位" が潜んでいた
- データの入手・変換で、何度もつまづいた
- 「見た目が微妙」を直すのが一番むずかしい ("良い" の言語化ができず作り直し多数)

AIは「速く形にする相棒」。
でも 舵は、人が握る。

一番の収穫は、成果物そのものより
「どこを自分がやるべきか」が見えたこと。

付録

成果物の概要

機能・技術・データ

主な機能

- 災害種別に応じた避難所ランキング（理由つき）
- 浸水域を避ける避難ルート／垂直避難の警告
- 現在地が危険なときの警告
- オフライン（PWA）／OIDC認証 + HTTPS

技術・データ

- Vanilla JS / MapLibre GL JS / Auth0(OIDC)
- Service Worker + IndexedDB
- 国土地理院（地図・避難所・標高）
- 国土数値情報（洪水浸水想定 A31）
- OpenRouteService（回避ルート）

公開URL <https://kyokoron.github.io/team-challenge/>